UI Test Case Tool

A full-stack web application for creating, managing, and organizing UI test cases as JSON files. Perfect for UI automation testing workflows where test cases need to be authored, validated, and exported for execution by testing frameworks like Playwright or Cypress.

Overview

This tool provides a user-friendly interface to:

- Create detailed test cases with multiple steps and validations
- Manage test execution order and enablement status
- Import/export test cases individually or in bulk
- Organize and search through large test suites
- Validate test case structure and requirements

Tech Stack

Frontend:

- React 18 with hooks
- Vite for development and bundling
- Tailwind CSS for styling
- Material Icons for UI elements
- Axios for API communication

Backend:

- Node.js with Express.js
- AJV for JSON schema validation
- File-based JSON storage
- CORS-enabled REST API

PROF

Features

Core Functionality

- **Single Test Creation**: Detailed form with comprehensive validation
- **Batch Test Creation**: Create multiple test cases in accordion-style interface
- **Drag & Drop Reordering**: Visual test case ordering with persistent `testOrder`
- **Keyboard Shortcuts**: Alt+↑/Alt+↓ for quick reordering
- **Search & Filter**: Find tests by description, order, or enabled status
- **Iterative Grouping**: Tag related steps with an `iterativeGroup` name to visually cluster them and prepare for grouped iterative execution logic

Import/Export

- **Drag & Drop Import**: Drop JSON files directly into the interface
- **File Picker Import**: Traditional file selection for JSON import
- **Individual Export**: Export single test cases as JSON
- **Bulk Export**: Export entire test suite as JSON array
- **Sample Data**: Includes example test cases for reference

Bulk Operations

- **Multi-select**: Select multiple test cases for bulk actions
- **Bulk Delete**: Remove multiple test cases at once
- **Bulk Duplicate**: Copy multiple test cases with auto-naming
- **Select All/Clear**: Quick selection management

Validation & Error Handling

- **Real-time Validation**: Form validation with immediate feedback
- **Schema Validation**: Server-side AJV schema enforcement
- **Duplicate Detection**: Prevents duplicate test orders
- **Required Field Indicators**: Clear marking of mandatory fields
- **Accessible Error Messages**: ARIA-compliant error reporting

User Experience

- **Toast Notifications**: Non-intrusive success/error messages
- **Responsive Design**: Works on desktop and mobile devices
- **Material Design Icons**: Consistent iconography throughout
- **Two-pane Layout**: Efficient space utilization
- **Keyboard Navigation**: Full keyboard accessibility support

Project Structure

PROF

```
ui-testcase-tool/
├── backend/                    # Express.js API server
│   ├── data/                   # JSON file storage
│   │   ├── *.json              # Individual test case files
│   ├── lib/
│   │   └── schema.js           # AJV validation schemas
│   ├── package.json
│   └── server.js               # Main server file
├── frontend/                   # React application
│   ├── dist/                   # Built application
│   ├── samples/                # Example files
│   │   ├── test-cases.json     # Sample test data
│   │   ├── action-executor.ts  # TypeScript examples
│   │   └── data-loader.ts      # TypeScript examples
│   ├── src/
│   │   ├── components/
│   │   │   ├── Dashboard.jsx   # Main application component
│   │   │   ├── TestCaseForm.jsx # Single test editor
```

```
│   │   │   ├── BatchCreate.jsx # Batch test creator
│   │   │   └── ToastProvider.jsx # Notification system
│   │   ├── api.js             # API client functions
│   │   └── main.jsx           # Application entry point
│   ├── index.html
│   ├── index.css              # Global styles
│   ├── package.json
│   ├── vite.config.ts         # Vite configuration
│   ├── tailwind.config.js     # Tailwind configuration
│   └── postcss.config.js      # PostCSS configuration
└── README.md
```

Prerequisites

- Node.js 18 or higher
- npm (comes with Node.js)

Installation & Setup

Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

2. Install dependencies:

```
npm install
```

3. Start the server:

```
npm start
# or for development with environment variables:
npm run dev
```

The backend will start on `http://localhost:4000` with CORS enabled.

Frontend Setup

1. Navigate to the frontend directory:

```
cd frontend
```

2. Install dependencies:

```
npm install
```

3. Start the development server:

```
npm start
```

The frontend will start on `http://localhost:5173` (or next available port).

Production Build

To build the frontend for production:

```
cd frontend
npm run build
```

To preview the production build:

```
npm run preview
```

The preview server runs on `http://localhost:4173` by default.

Usage Guide

Interface Layout

PROF

- **Left Panel**: Test case list with search, filters, and bulk actions
- **Right Panel**: Test case editor (Single mode) or batch creator (Batch mode)
- **Top Toolbar**: Mode switching, import/export, and global actions

Creating Test Cases

Single Mode

1. Click the "Single" mode button in the toolbar
2. Click the "+" button to create a new test case
3. Fill in the test description and configure settings
4. Add test steps with actions and validations
5. Click "Save" to store the test case

Batch Mode

1. Click the "Batch" mode button in the toolbar
2. Use the accordion interface to create multiple test cases
3. Each test case can have multiple steps and validations
4. Click "Save All" to store all test cases at once

Test Step Actions

Action	Required Fields	Description
`goto`	`path`	Navigate to a URL or path
`click`	`selector`	Click an element
`fill`	`selector`, `data`	Fill an input field
`type`	`selector`, `data`	Type text into an element
`press`	`selector`, `data`	Press keyboard keys
`hover`	`selector`	Hover over an element
`waitForTimeout`	`waitTime`	Wait for specified milliseconds
`custom`	`customName`	Execute custom test logic

Iterative Grouping

Use the optional `iterativeGroup` string on any step to associate multiple consecutive (or non-consecutive) steps into a logical group. In the UI:

- Steps sharing the same `iterativeGroup` are color-accented.
- A group badge appears in the step summary.
- The field is optional; leave it blank for ungrouped steps.

Planned (future) runtime behavior (not yet implemented): engine can iterate over matched elements once per group rather than per individual step, ensuring grouped actions execute in sequence for each element.

PROF

Validation Types

Type	Required Fields	Description
`toBeVisible`	`selector`	Element should be visible
`toBeHidden`	`selector`	Element should be hidden
`toHaveTitle`	`data`	Page should have specific title
`toHaveURL`	`data`	Page should have specific URL
`toHaveText`	`selector`, `data`	Element should contain text
`toHaveValue`	`selector`, `data`	Input should have specific value

Type	Required Fields	Description
`toHaveAttribute`	`selector`, `attribute`, `data`	Element should have attribute
`toHaveCSS`	`selector`, `cssProperty`, `data`	Element should have CSS property
`toHaveClass`	`selector`, `data`	Element should have CSS class

Import/Export

Importing

- **File Picker**: Click the import button and select JSON file(s)
- **Drag & Drop**: Drag JSON files into the drop zone
- **Supported Formats**: Single test object or array of test objects

Exporting

- **Single Test**: Click the export button on any test case
- **All Tests**: Click the "Export All" button in the toolbar
- **Format**: JSON with proper formatting and indentation

API Reference

Base URL: `http://localhost:4000/api`

Endpoints

GET /testcases

Retrieve all test cases, sorted by `testOrder`.

Response:

PROF

```
[
  {
    "filename": "example_test.json",
    "description": "Example Test Case",
    "enabled": true,
    "testOrder": 1,
    "testSteps": [...]
  }
]
```

POST /testcases

Create one or more test cases.

Request Body:

- Single test case object
- Array of test case objects

Response:

```
{
  "saved": [
    {
      "filename": "new_test.json",
      "description": "New Test",
      ...
    }
  ]
}
```

PUT /testcases/:filename

Update an existing test case.

Request Body: Test case object (without filename)

Response:

```
{ "ok": true }
```

DELETE /testcases/:filename

Delete a test case.

Response:

PROF

```
{ "ok": true }
```

Error Responses

All endpoints return appropriate HTTP status codes with error details:

```
{
  "error": "Error description",
  "details": {
    "duplicatesWithin": [1, 2],
    "conflictsWithExisting": [3, 4]
  }
}
```

Data Schema

Test Case Structure

```
{
  "description": "string (required)",
  "enabled": "boolean (default: true)",
  "testOrder": "number (auto-assigned if missing)",
  "testSteps": [
    {
      "stepName": "string (optional)",
      "action": "string (required)",
      "selector": "string (conditional)",
      "selectorType": "css|xpath|id|text|testId",
      "path": "string (conditional)",
      "data": "any (conditional)",
      "waitTime": "number (conditional)",
      "iterate": "boolean (default: false)",
  "iterativeGroup": "string (optional, groups related steps)",
      "customName": "string (conditional)",
      "soft": "boolean (default: false)",
      "validations": [
        {
          "type": "string (required)",
          "selector": "string (conditional)",
          "selectorType": "css|xpath|id|text|testId",
          "path": "string (optional)",
          "data": "any (conditional)",
          "message": "string (optional)",
          "soft": "boolean (default: false)",
          "attribute": "string (conditional)",
          "cssProperty": "string (conditional)"
        }
      ]
    }
  ]
}
```

PROF

Field Requirements

Test Step Actions:

- `goto`: Requires `path`
- `click`, `hover`: Require `selector`
- `fill`, `type`, `press`: Require `selector` and `data`
- `waitForTimeout`: Requires `waitTime` (>= 0)
- `custom`: Requires `customName`

Validations:

- All validations require `type`
- Most validations require `selector`
- `toHaveText`, `toHaveValue`, `toHaveTitle`, `toHaveURL`: Require `data`
- `toHaveAttribute`: Requires `attribute` and `data`
- `toHaveCSS`: Requires `cssProperty` and `data`

Sample Data

Example test case structure (see `frontend/samples/test-cases.json`):

```
{
  "description": "Accept Cookie Scenario and onboarding",
  "testOrder": 2,
  "enabled": true,
  "testSteps": [
    {
      "stepName": "Navigate to Proofing page",
      "action": "goto",
      "path": "/article/${TOKEN}"
    },
    {
      "stepName": "Accepting All Cookies",
      "action": "click",
      "selector": ".space-x-3>button:nth-child(3)",
      "iterativeGroup": "cookie-flow",
      "soft": true,
      "validations": [
        {
          "type": "toBeHidden",
          "selector": ".space-x-3>button:nth-child(3)",
          "message": "Accept Cookies button should be hidden after
accepting cookies",
          "soft": true
        }
      ]
    }
  ]
}
```

PROF

Storage

- Test cases are stored as individual JSON files in `backend/data/`
- Filenames are automatically generated from test descriptions
- Duplicate filenames get numeric suffixes (e.g., `test_1.json`, `test_2.json`)
- Files can be manually edited or backed up as standard JSON

Troubleshooting

Common Issues

Duplicate testOrder errors:

- Remove conflicting `testOrder` values or set them to `null`
- Server will auto-assign available order numbers

Import failures:

- Validate JSON syntax using a JSON validator
- Check that all required fields are present
- Review validation error details in the UI
- If failures mention validation, inspect step requirements (e.g., missing selector) or remove conflicting `testOrder` values
- If importing samples containing `_id`, `createdAt`, `updatedAt`, or existing `filename`, the server now strips these automatically

Styling issues:

- Ensure Tailwind CSS is properly configured
- Verify that the Vite dev server is running
- Check browser console for CSS loading errors

API connection issues:

- Verify backend server is running on port 4000
- Check for CORS issues in browser console
- Ensure no firewall blocking localhost connections

Data Reset

To reset all test data:

1. Stop the backend server
2. Remove all files from `backend/data/` directory
3. Restart the backend server

PROF

Accessibility

- **ARIA Labels**: All interactive elements have appropriate labels
- **Keyboard Navigation**: Full keyboard support for all functionality
- **Screen Reader Support**: Semantic HTML and ARIA attributes
- **High Contrast**: Sufficient color contrast ratios
- **Focus Management**: Clear focus indicators and logical tab order

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add some amazing feature'`)

4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

Development Guidelines

- Follow existing code style and patterns
- Add appropriate error handling
- Include validation for new features
- Update documentation for API changes
- Test on multiple browsers and screen sizes

Future Enhancements

- [x] Test execution integration with Playwright/Cypress
- [ ] Test case templates and snippets
- [ ] Test suite organization with folders/tags
- [ ] User authentication and workspace isolation
- [ ] Real-time collaboration features
- [ ] Test case versioning and history
- [ ] Advanced search and filtering options
- [ ] Test case scheduling and automation
- [ ] Performance monitoring and analytics
- [ ] Plugin system for custom actions/validations

License

This project is provided as-is without a specific license. Please contact the project maintainers for licensing information.